

ksat-ports

Build & Shadow-Test Setup (MicroSAT)

How this project is built and run

Contents

Gradle build	3
DIMACS parser	3
Shadow test	3
Broader test CNFs + benchmark	3
Bug fixed during the port	4
Running it	4

Gradle build

Multi-module KMP library (no Compose, no Android): root `settings.gradle.kts` includes `:ksat-common` and `:microsat`. Version baseline from `luckySweep/kmp/gradle/libs.versions.toml` (Kotlin 2.2.20); this repo's own `gradle/libs.versions.toml` holds only what the library needs (kotlin + kotlin-test). Targets per module: `jvm()`, `iosArm64()`, `iosSimulatorArm64()`. `:microsat` depends on `:ksat-common`. `gradle.properties` caps the daemon at `-Xmx1536M` (16 GB Mac). Gradle wrapper pinned to 9.2.1.

DIMACS parser

`ksat-common/.../DimacsCnf.kt` parses CNF text (skip `c` lines, read `p cnf n m`, literals until `0`), matching MicroSAT's file parser, and has `isSatisfiedBy` to check a model against a CNF. Tests embed CNF text as strings (commonTest needs no file access).

Shadow test

`microsat/src/commonTest/.../ShadowTraceTest.kt` runs MicroSat with a trace sink and compares the trace line-for-line against the golden C reference trace (`shadow/microsat-c/microsat_trace.c`, built `cc -O2`, run with `LSTRACE=1`). Asserts verdict match and, on SAT, that the model satisfies the CNF.

CNFs covered (in `shadow/cnf/`; regenerate golden traces with `shadow/build_and_trace.sh --all`):

- `sat_small` — SAT, one conflict during search
- `unsat_small` — UNSAT via conflicting units at parse time
- `conflict_small` — conflict analysis then root-level UNSAT
- `conflict_sat` — conflict during search, backtrack, then a model
- `php_3_2` — pigeonhole 3/2, UNSAT with conflict

Known gap now closed

RESTART/REDUCE are not triggered by these small embedded instances. They are now covered by a file-driven JVM shadow test on generated harder CNFs — see the next section.

Broader test CNFs + benchmark

New durable tooling under `shadow/tools/` (all checked in, re-runnable):

- `gen_cnf.py` — deterministic (seeded) CNF generator. Writes into `shadow/cnf/`: tiny SAT/UNSAT extras, random 3-SAT at the phase transition (20 & 50 vars, ratio 4.26), pigeonhole `php_(n+1)_n` for `n = 3..7` (all UNSAT, force conflicts), and `hard_rand_unsat_60` (60 vars, ratio 5.5) aimed at forcing enough conflicts + lemmas to trigger RESTART and REDUCE.
- `regen_golden.sh` — generates the CNFs, builds the instrumented C reference (`cc -O2`), runs each CNF with `LSTRACE=1`, strips the `s /c` status lines, and writes the pure event trace to `shadow/golden/<name>.trace` plus a `manifest.txt` marking which CNFs contain RESTART / REDUCE lines.
- `benchmark.sh` — builds the C original (`cc -O2 microsat_orig.c`), times it on every CNF (median of `N` runs, whole process), runs the Kotlin benchmark (`./gradlew :microsat:runBenchmark, solve-only, warmed up`), and prints a side-by-side C-vs-Kotlin table.

File-driven test: `microsat/src/jvmTest/.../ShadowTraceFilesTest.kt` compares the Kotlin trace of every `shadow/cnf/<name>.cnf` (that has a `shadow/golden/<name>.trace`) line-for-line against the C golden trace, checks the verdict, and — on SAT — that the model satisfies the CNF. A second test asserts at least one generated instance actually drives RESTART **and** REDUCE, so those paths stay covered. Both skip gracefully when `shadow/golden/` has not been generated (fresh checkout stays green). The Kotlin benchmark lives in `microsat/src/jvmMain/.../Benchmark.kt` (Gradle task `runBenchmark`).

RESTART/REDUCE port review

An independent line-by-line audit of `restart()`, `reduceDB()`, their call site in `solveInternal()`, and `addClause` (re-adding surviving lemmas) found no discrepancies vs the C, high confidence. Only cross-file difference: `MEM_MAX = 1 shl 24` in Kotlin vs `1 << 30` in C — a capacity choice (Kotlin throws on overflow, C `exit(0)`), not a logic change. For the generated instances the lemma DB stays far under 16M ints even after REDUCE.

Bug fixed during the port

`reduceDB` declared `var i` twice in one function scope (C reuses one for index across two loops; Kotlin rejects it). Renamed the second index. This was a compile error, not a trace mismatch — after the fix all shadow tests passed and every Kotlin trace matched the C reference byte-for-byte.

Running it

- Generate CNFs + C golden traces: `bash shadow/tools/regen_golden.sh`
- C reference traces (original 5): `shadow/build_and_trace.sh --all`
- Kotlin shadow tests: `./gradlew :microsat:jvmTest`
- Runtime benchmark (C vs Kotlin): `bash shadow/tools/benchmark.sh`